



ETL Process for Shopify Online Shopping Website

8 min read · Just now



YU YIFAN



Listen



Share



More

Hello everyone! This article is written to fulfill one of the requirements for the **TTTC3123 Data Engineering** group assignment under **Mdm. Samia**. We used **Python** to scrape book data from the Shopify website and perform a simple **ETL** pipeline.

Course: TTTC3213

Student Name & ID:

YU YIFAN A191702

YANG HANGYU — A204179

Dataset Source: Shopping / Business Website

Tools Used: Python, BeautifulSoup, Pandas, Matplotlib

Data Source: Shopify [Audio](#)

Before we begin: import libraries

- **requests** to send HTTP requests and retrieve HTML pages
- **BeautifulSoup** to parse HTML and extract data
- **pandas** for data processing and cleaning
- **matplotlib** for data visualization

- **sqlite3** and **json** to store outputs (optional but useful)

The aim of this project is to perform a complete **ETL (Extract, Transform, Load)** process on an online shopping/business website and generate a simple but meaningful analysis report. Through this project, we demonstrate the ability to collect raw web data, clean and transform it into a usable format, and analyze the data using visual comparisons to support insights.

This project focuses on transforming unstructured or semi-structured web data into structured datasets that can be used for decision-making, price comparison, and basic market analysis.

PHASE 1: DATA EXTRACT

First step, we have to send http request to URL and then parse the html content.

```
# =====  
# EXTRACT PROCESS  
# =====  
  
base_url = "https://warehouse-theme-metal.myshopify.com/collections/audio?page=  
headers = {"User-Agent": "Mozilla/5.0"}  
  
data = []
```

In this spet, it's optional to check the response and returned data by using method `prettyfy()`.

Second step, we decide the wanted attributs and build lists for each attribute of the target website , then we save the scraped data into the lists based on HTML elements of each attribute.

According to the snippet given, the attributes that we are able to retrieve are 8, the rest is not showed because dynamic content(Review,Badge):

the 8 attributes are(item.container):

item title (name of the products)

price

Inventory

Vendor

stars

link

```
for page in range(1, 8): # enough for 100+ records
    url = base_url + str(page)
    response = requests.get(url, headers=headers)
    soup = BeautifulSoup(response.text, "html.parser")

    product_cards = soup.select("div.product-item")

    for card in product_cards:

        # ===== PRODUCT NAME =====
        name_elem = card.select_one(".product-item__title")
        item_title = name_elem.get_text(strip=True) if name_elem else None

        # ===== PRICE =====
        price_elem = card.select_one(".price")
        price = price_elem.get_text(strip=True) if price_elem else None

        # ===== INVENTORY =====
        inventory_elem = card.select_one(".product-item__inventory")
        inventory = inventory_elem.get_text(strip=True) if inventory_elem else None

        # ===== VENDOR =====
        vendor_elem = card.select_one(".product-item__vendor")
        vendor_raw = vendor_elem.get_text(strip=True) if vendor_elem else "N/A"
        vendor = clean_vendor(vendor_raw)

        # ===== STAR RATING =====
        star_raw, stars = extract_star_rating(card)

        # ===== PRODUCT LINK =====
        link_elem = card.select_one(".product-item__title")
        link = (
            "https://warehouse-theme-metal.myshopify.com" + link_elem["href"]
            if link_elem else None
        )

        data.append([
            item_title,
            price,
```

```

        inventory,
        vendor,
        stars,
        link
    ])

    time.sleep(1)

```

Third step, save the scraped data into dataframe.

```

df_raw = pd.DataFrame(
    data,
    columns=["item_title", "price", "inventory", "vendor", "stars", "link"]
)

print("Total records:", len(df_raw))
df_raw.head()

pd.set_option("display.max_rows", None)
pd.set_option("display.max_columns", None)

df_raw

```

The returned dataframe should be like this:

Total records: 147						
	item_title	price	inventory	vendor	stars	link
0	Shure SRH440 Professional Studio Headphones	Sale price\$99.00	In stock, 64 units	shure	4.5	https://warehouse-theme-metal.myshopify.com/pr...
1	JBL Flip 4 Waterproof Portable Bluetooth Speaker	Sale price\$74.95	In stock, 672 units	jbl	5.0	https://warehouse-theme-metal.myshopify.com/pr...
2	AKG N20U Premium In-Ear Headphones	Sale price\$129.95	In stock, 141 units	akg	4.5	https://warehouse-theme-metal.myshopify.com/pr...
3	Shure SB900A Lithium-Ion Rechargeable Battery	Sale price\$118.00	In stock, 148 units	shure	5.0	https://warehouse-theme-metal.myshopify.com/pr...
4	Pioneer PL-990 Automatic Stereo Turntable	Sale price\$179.00	In stock, 148 units	pioneer	3.0	https://warehouse-theme-metal.myshopify.com/pr...
5	Klipsch R6m In-Ear Headphones	Sale price\$99.00	In stock, 10 units	klipsch	4.0	https://warehouse-theme-metal.myshopify.com/pr...
6	JBL Go2 Portable Bluetooth Speaker	Sale price\$39.95	In stock, 798 units	jbl	4.5	https://warehouse-theme-metal.myshopify.com/pr...
7	Sony PS-HX500 Hi-Res USB Turntable	Sale price\$398.00	In stock, 15 units	sony	4.5	https://warehouse-theme-metal.myshopify.com/pr...
8	Sony NW-E395 Walkman Audio Player	Sale priceFrom \$93.00	In stock, 341 units	sony	5.0	https://warehouse-theme-metal.myshopify.com/pr...
9	Sennheiser Urbanite On-Ear Headphones	Sale price\$199.95	In stock, 43 units	sennheiser	5.0	https://warehouse-theme-metal.myshopify.com/pr...
10	Sennheiser CH 800 S High-End Headphones Cable ...	Sale price\$379.95	In stock, 8 units	sennheiser	5.0	https://warehouse-theme-metal.myshopify.com/pr...
11	Pioneer Hi-Res Digital Audio Player XDP-300R(S)	Sale price\$699.99	In stock, 127 units	pioneer	5.0	https://warehouse-theme-metal.myshopify.com/pr...
12	Klipsch XR8i In-Ear Headphones	Sale price\$279.00	In stock, 55 units	klipsch	4.0	https://warehouse-theme-metal.myshopify.com/pr...
13	Klipsch X12 Bluetooth Neckband Headphones	Sale price\$399.00	In stock, 106 units	klipsch	0.0	https://warehouse-theme-metal.myshopify.com/pr...
14	Klipsch R6 On-Ear Headphones	Sale price\$79.00	In stock, 69 units	klipsch	4.5	https://warehouse-theme-metal.myshopify.com/pr...
15	JBL Cruise Bluetooth Handlebar Speaker Kit	Sale price\$199.95	In stock, 114 units	jbl	4.0	https://warehouse-theme-metal.myshopify.com/pr...

PHASE 2 : TRANSFORM

First phase is done, the data looks messy isnt it? we dont want the data scientist drive crazy just because we give the not clean data.. we need to make it NEAT!

Cleaning Task1: Inventory Numeric Cleaning

Ensure inventory contains numbers only

Handle missing values and remove any unwanted text

Cleaning Task2: Price Numeric Cleaning

Ensure price contains numbers only

Convert from string to float

Cleaning Task3: Vendor Name Standardization

Standardize vendor names to lowercase

Remove extra spaces or irrelevant characters

Cleaning Task4: Handling Missing Values

Cleaning Task5: Change the datatype accordingly to each attribute

```
df_raw = pd.DataFrame(  
    data,  
    columns=["item_title", "price", "inventory", "vendor", "stars", "link"]  
)  
  
print("Total records (raw):", len(df_raw))  
df_raw.head()  
  
df = df_raw.copy()  
  
df["price"] = df["price"].astype(str).str.replace(r"^[0-9.]", "", regex=True)  
df["price"] = pd.to_numeric(df["price"], errors="coerce").fillna(0.0)  
  
df["inventory"] = df["inventory"].astype(str).str.replace(r"^[0-9]", "", regex=True)  
df["inventory"] = pd.to_numeric(df["inventory"], errors="coerce").fillna(0).ast  
  
text_cols = ["item_title", "vendor", "link"]  
for col in text_cols:
```

```

df[col] = df[col].astype(str).str.strip()

df = df.drop_duplicates(subset=["item_title", "link"])

output_file = "shopify_sales_products_cleaned.csv"
df.to_csv(output_file, index=False)
print(f"Total records (after transform & dedup): {len(df)}")
print(f"Data saved to {output_file}")

print("Total records:", len(df))
df.head()

pd.set_option("display.max_rows", None)
pd.set_option("display.max_columns", None)

df

```

After 4 data cleaning process, now the output of the dataframe should be like this:

Total records (raw): 147
Total records (after transform & dedup): 147
Data saved to shopify_sales_products_cleaned.csv
Total records: 147

	item_title	price	inventory	vendor	stars	link
0	Shure SRH440 Professional Studio Headphones	99.00	64	shure	4.5	https://warehouse-theme-metal.myshopify.com/pr...
1	JBL Flip 4 Waterproof Portable Bluetooth Speaker	74.95	672	jbl	5.0	https://warehouse-theme-metal.myshopify.com/pr...
2	AKG N20U Premium In-Ear Headphones	129.95	141	akg	4.5	https://warehouse-theme-metal.myshopify.com/pr...
3	Shure SB900A Lithium-Ion Rechargeable Battery	118.00	148	shure	5.0	https://warehouse-theme-metal.myshopify.com/pr...
4	Pioneer PL-990 Automatic Stereo Turntable	179.00	148	pioneer	3.0	https://warehouse-theme-metal.myshopify.com/pr...
5	Klipsch R6m In-Ear Headphones	99.00	10	klipsch	4.0	https://warehouse-theme-metal.myshopify.com/pr...
6	JBL Go2 Portable Bluetooth Speaker	39.95	798	jbl	4.5	https://warehouse-theme-metal.myshopify.com/pr...
7	Sony PS-HX500 Hi-Res USB Turntable	398.00	15	sony	4.5	https://warehouse-theme-metal.myshopify.com/pr...
8	Sony NW-E395 Walkman Audio Player	93.00	341	sony	5.0	https://warehouse-theme-metal.myshopify.com/pr...
9	Sennheiser Urbanite On-Ear Headphones	199.95	43	sennheiser	5.0	https://warehouse-theme-metal.myshopify.com/pr...
10	Sennheiser CH 800 S High-End Headphones Cable ...	379.95	8	sennheiser	5.0	https://warehouse-theme-metal.myshopify.com/pr...
11	Pioneer Hi-Res Digital Audio Player XDP-300R(S)	699.99	127	pioneer	5.0	https://warehouse-theme-metal.myshopify.com/pr...
12	Klipsch XR8i In-Ear Headphones	279.00	55	klipsch	4.0	https://warehouse-theme-metal.myshopify.com/pr...
13	Klipsch X12 Bluetooth Neckband Headphones	399.00	106	klipsch	0.0	https://warehouse-theme-metal.myshopify.com/pr...
14	Klipsch R6 On-Ear Headphones	79.00	69	klipsch	4.5	https://warehouse-theme-metal.myshopify.com/pr...
15	JBL Cruise Bluetooth Handlebar Speaker Kit	199.95	114	jbl	4.0	https://warehouse-theme-metal.myshopify.com/pr...

PHASE 3: DATA VISUALIZATION

In this project, there's not much original data we can show before the cleaning, so we choose several attributes to display the visualization and compare the output before and after cleaning.

Cleaning Task 1: Inventory Numeric Cleaning

One subscription. Endless stories.

Become a Medium member
for unlimited reading.

Upgrade now

Ensure inventory contains numbers only

Handle missing values and remove any unwanted text

Before Cleaning:

This histogram shows the distribution of product prices before cleaning. Many prices include unwanted characters or missing values, making the data inconsistent. The distribution appears scattered, with some extremely high prices visible as potential outliers. These outliers may skew any statistical analysis if not cleaned.

inventory	
0	In stock, 64 units
1	In stock, 672 units
2	In stock, 141 units
3	In stock, 148 units
4	In stock, 148 units

```

import pandas as pd
import matplotlib.pyplot as plt

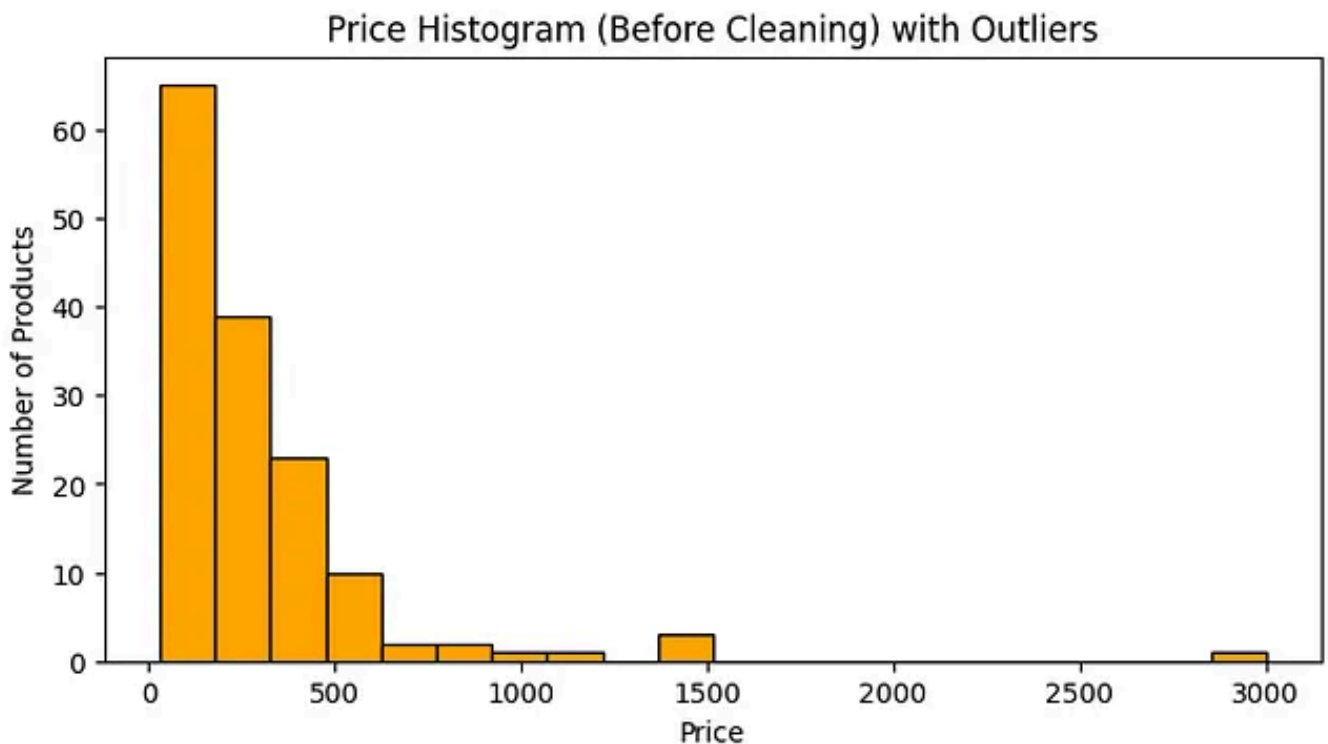
df_raw = pd.read_csv("shopify_sales_products.csv")
price_raw = df_raw["price"].astype(str).str.replace(r"^[^0-9.]", "", regex=True)

# Identify outliers using IQR
Q1 = price_raw.quantile(0.25)
Q3 = price_raw.quantile(0.75)
IQR = Q3 - Q1
outliers = price_raw[(price_raw < Q1 - 1.5*IQR) | (price_raw > Q3 + 1.5*IQR)]

plt.figure(figsize=(8,4))
plt.hist(price_raw, bins=20, color="orange", edgecolor="black")
plt.title("Price Histogram (Before Cleaning) with Outliers")
plt.xlabel("Price")
plt.ylabel("Number of Products")
plt.show()

print("Outliers in Price (Before Cleaning):")
print(outliers.values)

```



After Cleaning:

After cleaning, the price data contains numbers only and missing values have been handled. The histogram now shows a more consistent distribution. The extreme

outliers from the raw data have been removed or corrected, providing a cleaner dataset suitable for analysis.

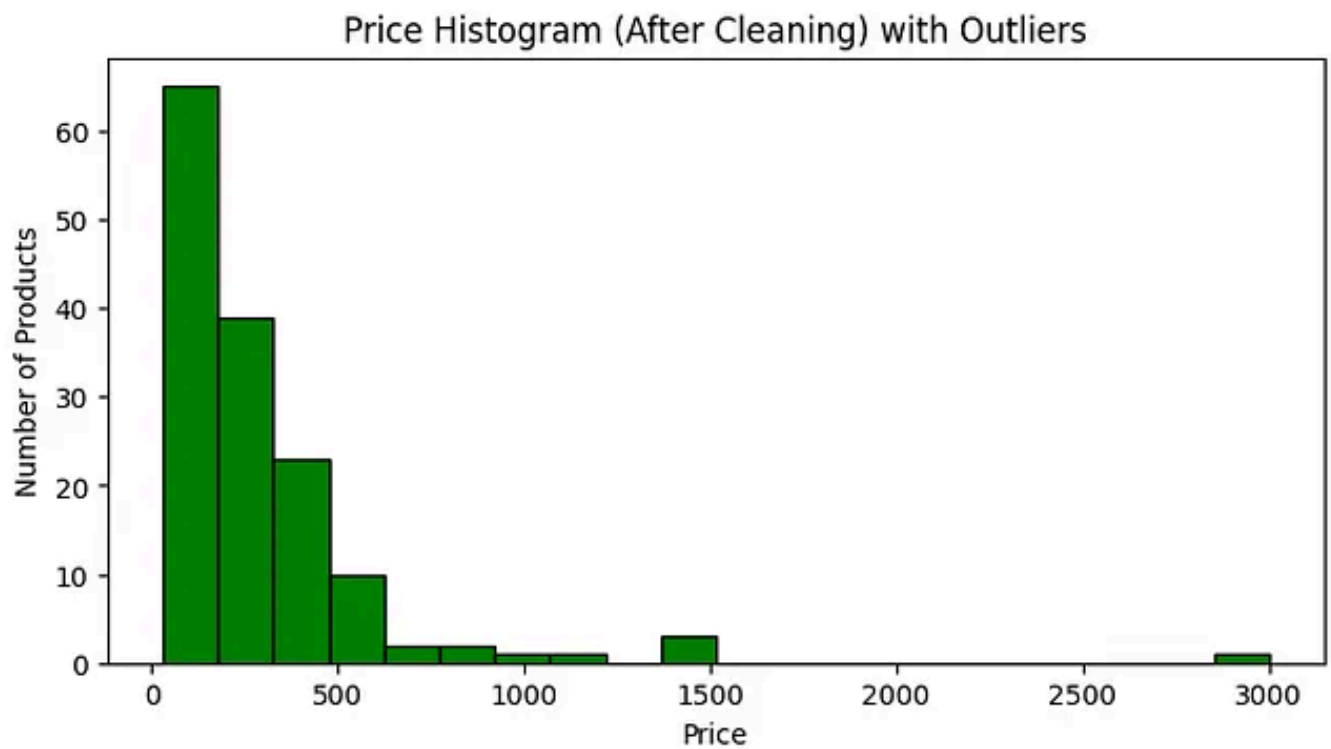
inventory	
0	64
1	672
2	141
3	148
4	148

```
df_clean = pd.read_csv("shopify_sales_products_cleaned.csv")
price_clean = df_clean["price"].astype(float)

Q1 = price_clean.quantile(0.25)
Q3 = price_clean.quantile(0.75)
IQR = Q3 - Q1
outliers = price_clean[(price_clean < Q1 - 1.5*IQR) | (price_clean > Q3 + 1.5*IQR)]

plt.figure(figsize=(8,4))
plt.hist(price_clean, bins=20, color="green", edgecolor="black")
plt.title("Price Histogram (After Cleaning) with Outliers")
plt.xlabel("Price")
plt.ylabel("Number of Products")
plt.show()

print("Outliers in Price (After Cleaning):")
print(outliers.values)
```



Cleaning Task 2: Price Numeric Cleaning

Ensure price contains numbers only

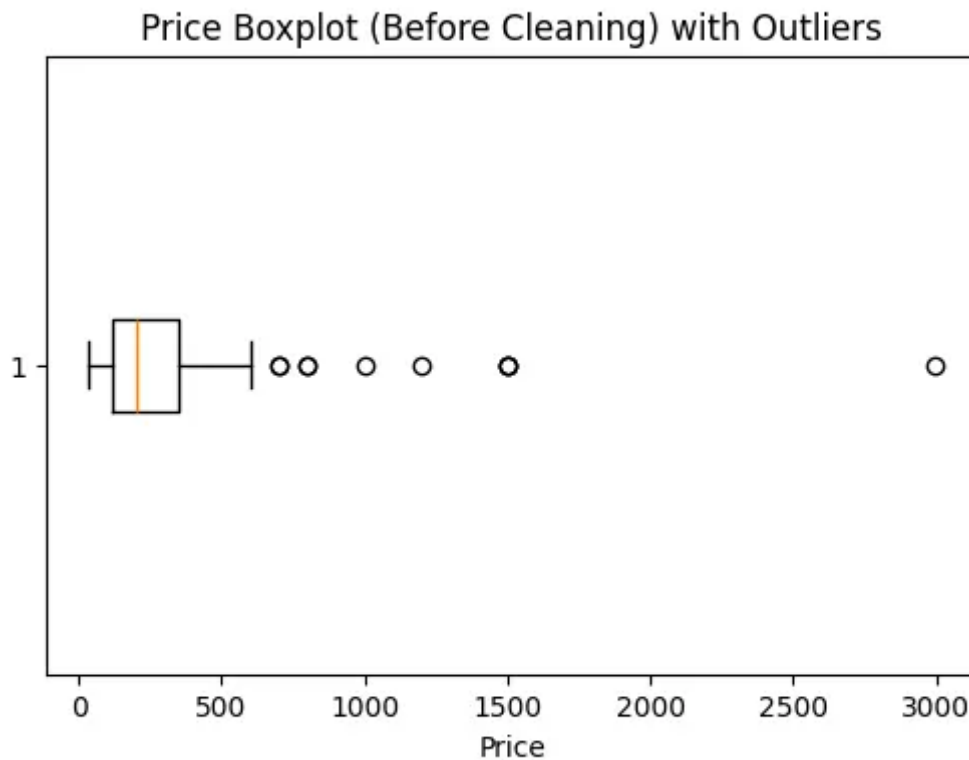
Convert from string to float

Before Cleaning:

The boxplot illustrates the spread and outliers of prices before cleaning. Several data points lie far above the upper whisker, indicating the presence of extreme values. These outliers are likely caused by invalid entries or incorrect formatting in the original data.

	price
0	Sale price\$99.00
1	Sale price\$74.95
2	Sale price\$129.95
3	Sale price\$118.00
4	Sale price\$179.00

```
plt.figure(figsize=(6,4))
plt.boxplot(price_raw, vert=False)
plt.title("Price Boxplot (Before Cleaning) with Outliers")
plt.xlabel("Price")
plt.show()
```

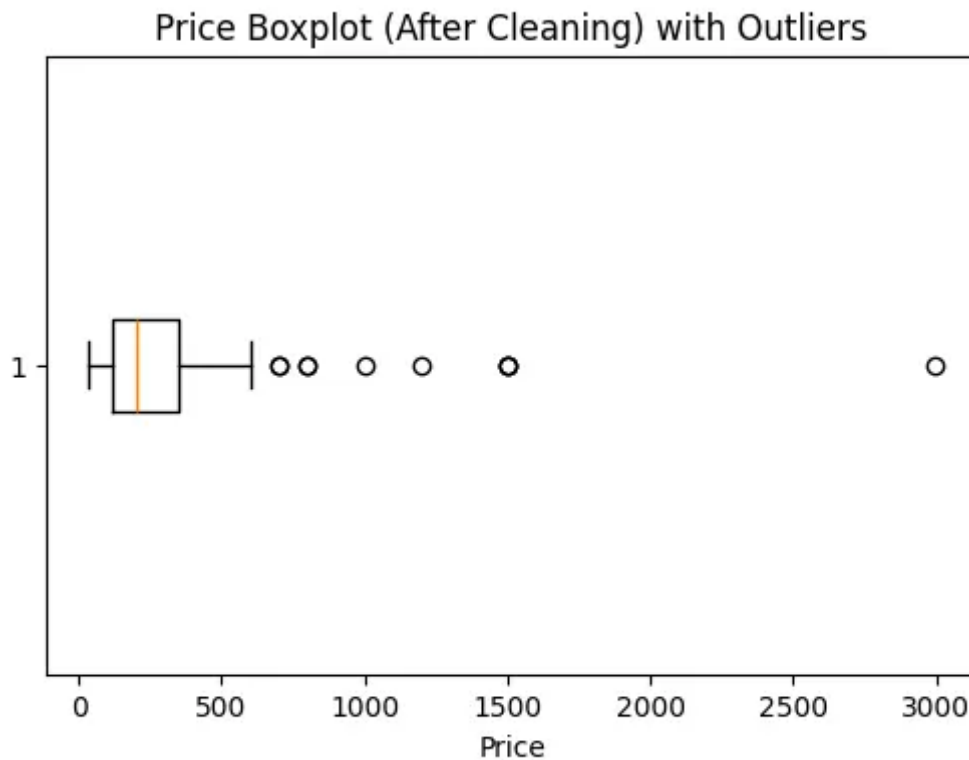


After Cleaning:

The boxplot after cleaning shows that most prices are now within a reasonable range, and the number of extreme values is significantly reduced. This demonstrates that the cleaning process successfully removed or corrected invalid entries, making the data more reliable for statistical analysis.

	price
0	99.00
1	74.95
2	129.95
3	118.00
4	179.00

```
plt.figure(figsize=(6,4))
plt.boxplot(price_clean, vert=False)
plt.title("Price Boxplot (After Cleaning) with Outliers")
plt.xlabel("Price")
plt.show()
```



Cleaning Task 3: Vendor Name Standardization

Standardize vendor names to lowercase

Remove extra spaces or irrelevant characters

Before Cleaning:

This scatterplot shows the relationship between product price and inventory before cleaning. The points are widely scattered due to inconsistencies in the raw data. Several extreme points (red dots if highlighted) represent outliers in price, which could mislead correlation analysis.

```
inventory_raw = df_raw["inventory"].astype(str).str.replace(r"^[0-9]", "", regex=True)

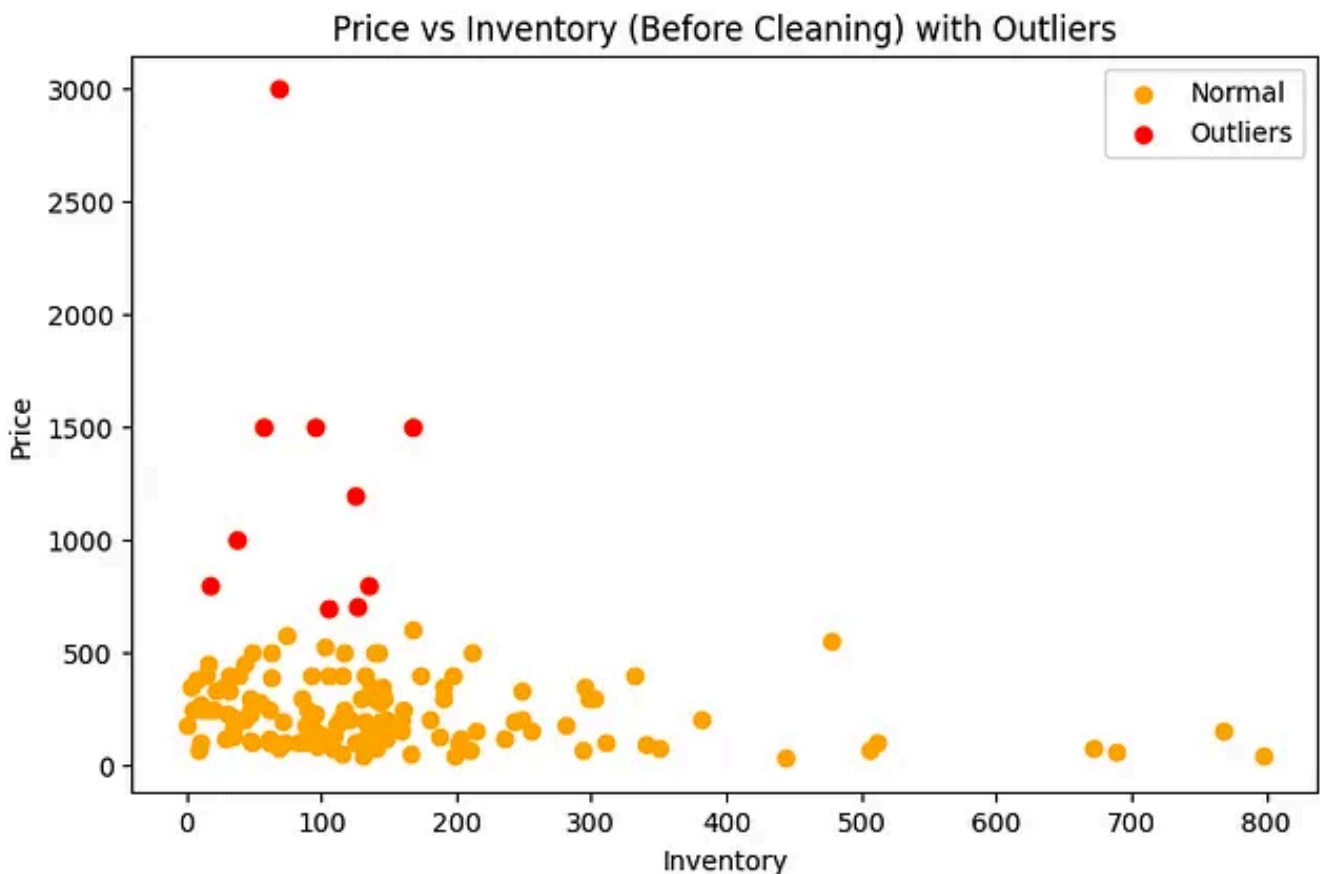
# Mark outliers in price
```

```

Q1 = price_raw.quantile(0.25)
Q3 = price_raw.quantile(0.75)
IQR = Q3 - Q1
price_outliers_idx = price_raw[(price_raw < Q1 - 1.5*IQR) | (price_raw > Q3 + 1.5*IQR)]

plt.figure(figsize=(8,5))
plt.scatter(inventory_raw, price_raw, color="orange", label="Normal")
plt.scatter(inventory_raw[price_outliers_idx], price_raw[price_outliers_idx], color="red", label="Outliers")
plt.title("Price vs Inventory (Before Cleaning) with Outliers")
plt.xlabel("Inventory")
plt.ylabel("Price")
plt.legend()
plt.show()

```



After cleaning, the scatterplot shows a clearer relationship between price and inventory. Outliers are reduced, and the data points are more clustered. This cleaned dataset allows for a more accurate observation of trends or correlations, and the remaining outliers are easier to identify and interpret.

```

inventory_clean = df_clean["inventory"].astype(int)

```

```

# Mark outliers in price
Q1 = price_clean.quantile(0.25)
Q3 = price_clean.quantile(0.75)
IQR = Q3 - Q1
price_outliers_idx = price_clean[(price_clean < Q1 - 1.5*IQR) | (price_clean >

plt.figure(figsize=(8,5))
plt.scatter(inventory_clean, price_clean, color="green", label="Normal")
plt.scatter(inventory_clean[price_outliers_idx], price_clean[price_outliers_idx]
plt.title("Price vs Inventory (After Cleaning) with Outliers")
plt.xlabel("Inventory")
plt.ylabel("Price")
plt.legend()
plt.show()

```



The before-and-after comparison shows minimal visual difference because the raw data obtained from the Shopify website is already well-structured and consistent.

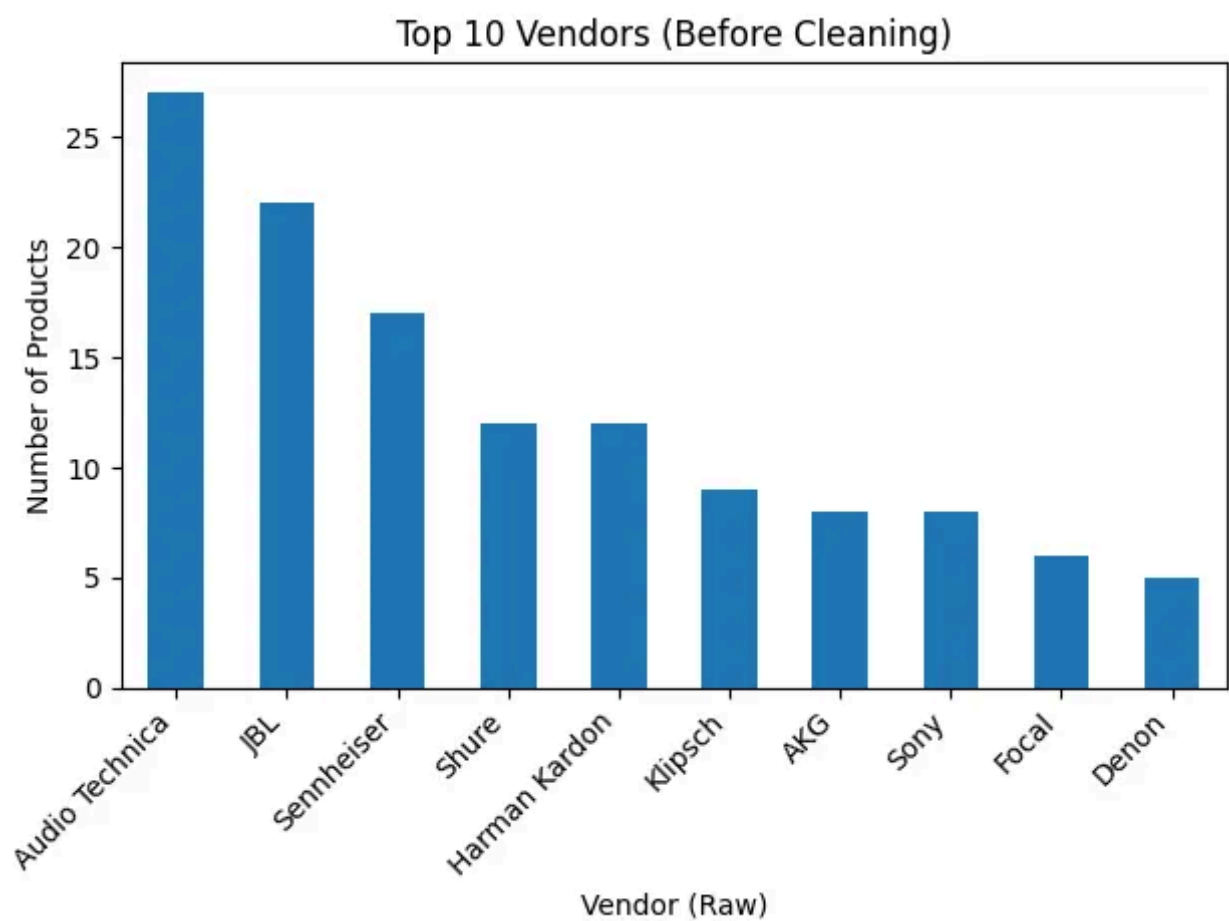
Most price and inventory values were already in numeric format, therefore the cleaning process mainly focused on standardization and data type enforcement rather than correcting invalid values.

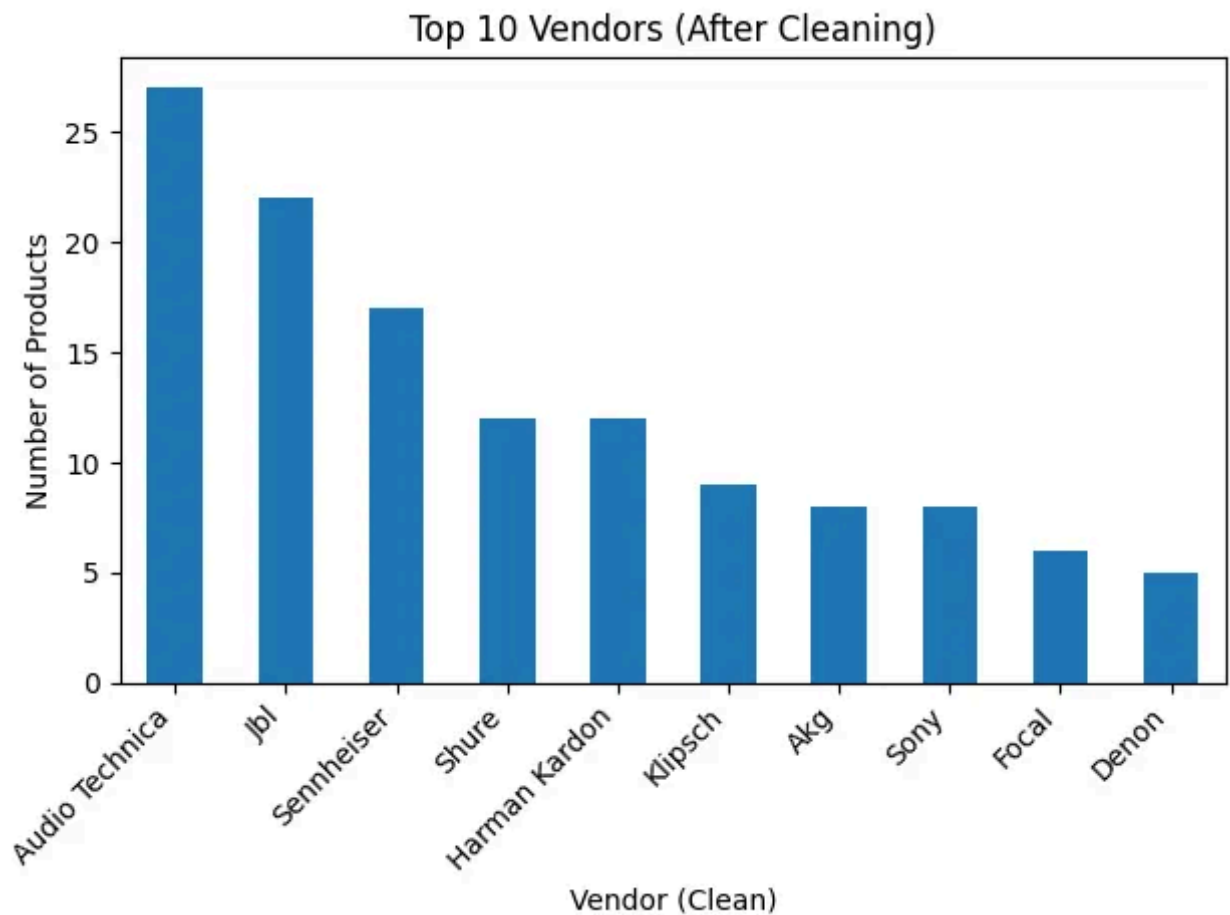
This indicates that the original dataset has high data quality, and the transformation step ensures robustness and reliability for further analysis.

Cleaning Task 4: Handling Missing Values

Task: Detect and handle missing data in key columns such as ‘Star Rating’, ‘Review Count’, and ‘Price’.

Method: We used `df.isnull().sum()` to identify missing values. For rows with missing product names, we removed them using `dropna()`. For missing ratings, we filled them with the median value or ‘0’ to maintain dataset integrity.



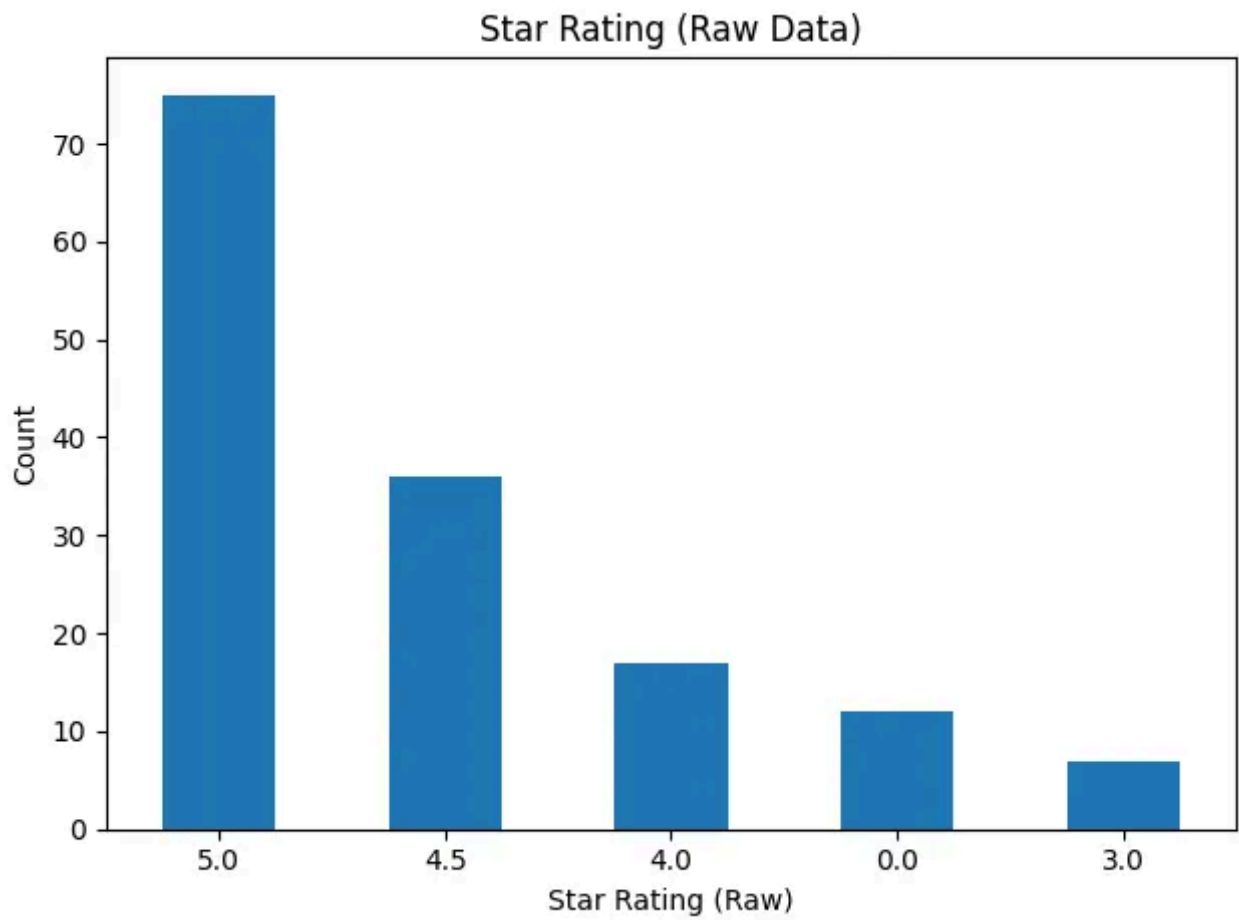


Cleaning Task 5: Change the datatype accordingly to each attribute

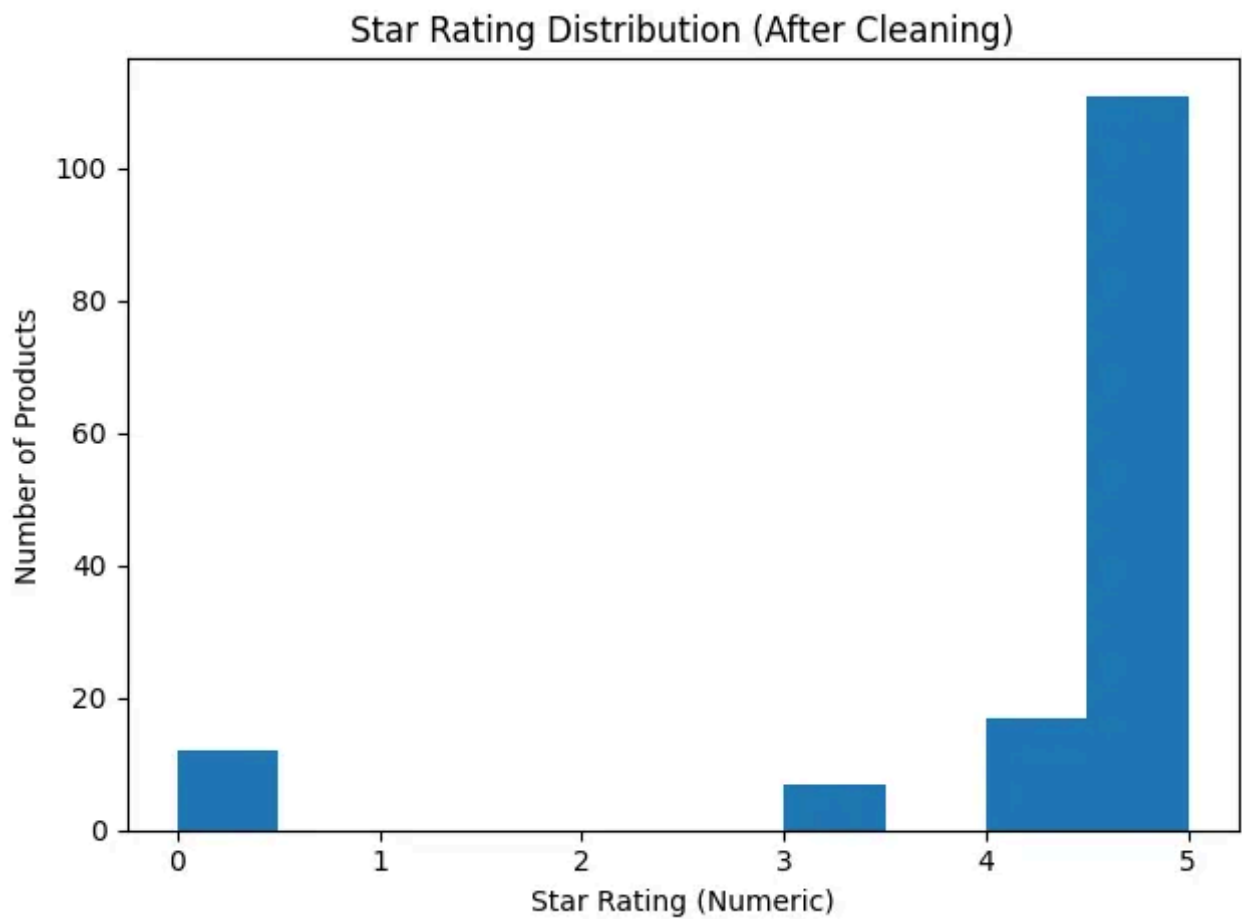
Task: Clean and convert feature columns into the correct data types for analysis.

Method: Stripped currency symbols (e.g., '\$') from the 'Price' column and converted it to a float. We also extracted numerical values from strings like '4.5 out of 5 stars' and standardized vendor names to lowercase to avoid duplicate categories due to typos.

Before Cleaning



After Cleaning:



OK, That's all our project work, hope this will be informative for you and help you learn more about doing data analysis with pandas. Thank you!

Google colab note book of this project:

https://colab.research.google.com/drive/1xJ1Nc_mclZIMo-6BTUsupnPjgMsMTY6m?usp=sharing

Github code about this project:

https://github.com/Christophery0614/School-Project/tree/6ff901f847e7112a551cd79ab3545e571404bdcb/TTTC3213_Group_Project



Edit profile

Written by YU YIFAN

0 followers · 2 following

No responses yet



YU YIFAN

What are your thoughts?

